

1. Verify if master is in clean state.

Ans: git status

2. Create a branch "**GitWork**". Add a file "hello.xml".

Ans: git checkout -b GitWork

echo "<message>Hello from Branch</message>" > hello.xml

3. Update the content of "hello.xml" and observe the status

Ans: git status

4. Commit the changes to reflect in the branch

Ans: git add hello.xml

git commit -m "Add hello.xml in GitWork branch"

5. Switch to master.

Ans: git checkout master

6. Add a file "**hello.xml**" to the master and add some different content than previous.

Ans: echo "<message>Hello from Master Trunk</message>" > hello.xml

7. Commit the changes to the master

Ans: git add hello.xml

git commit -m "Add conflicting hello.xml in master"

8. Observe the log by executing "**git log --oneline --graph --decorate --all**"

Ans: git log --oneline --graph --decorate --all

9. Check the differences with Git diff tool

Ans: git diff master GitWork

10. For better visualization, use P4Merge tool to list out all the differences between master and branch

Ans: git difftool master GitWork

11. Merge the bran to the master

Ans: git merge GitWork

12. Observe the git mark up.

Ans: cat hello.xml

13. Use 3-way merge tool to resolve the conflict

Ans: git mergetool

14. Commit the changes to the master, once done with conflict

Ans: git commit -m "Merge branch GitWork and resolve conflict in hello.xml"

15. Observe the git status and add backup file to the .gitignore file.

Ans: git status

echo "*.orig" >> .gitignore

16. Commit the changes to the .gitignore

Ans: git add .gitignore

git commit -m "Add merge tool backup extensions to gitignore"

17. List out all the available branches

Ans: git branch

18. Delete the branch, which merge to master.

Ans: git branch -d GitWork

19. Observe the log by executing “**git log --oneline --graph --decorate**”

Ans: git log --oneline --graph --decorate